

Práctica: Clasificación de Noticias con Deep Learning

El objetivo es el mismo: clasificar noticias españolas, pero esta vez usaremos una **Red Neuronal**.

Mientras que el "Machine Learning clásico" usa recuento de palabras (Bag of Words), el **Deep Learning** permite que el modelo entienda el contexto y la relación entre palabras mediante **Embeddings** (vectores numéricos densos).

Para esta práctica usaremos **TensorFlow/Keras**, que es el estándar más amigable para empezar en Deep Learning.

Bloque 1: Instalación y Preparación

A diferencia de Scikit-learn, aquí necesitamos herramientas para procesar tensores.

```
# Instala la librería si no la tienes
# pip install tensorflow pandas scikit-learn

import pandas as pd
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Cargamos los datos (asumiendo el mismo CSV de la práctica anterior)
df = pd.read_csv('nombre.csv', encoding='UTF-8')

# Convertimos las etiquetas de texto (ej: "deportes") a números (0, 1, 2...)
le = LabelEncoder()
df['label_num'] = le.fit_transform(df['type'])
num_clases = len(le.classes_)
```

Bloque 2: Tokenización y Padding (El "corazón" del DL en texto)

En Deep Learning no usamos `CountVectorizer`. Convertimos cada palabra en un índice único y hacemos que todas las noticias tengan la **misma longitud** añadiendo ceros (Padding).

```
# Parámetros de configuración
vocab_size = 10000 # Solo usamos las 10,000 palabras más comunes
max_length = 200   # Cortamos o rellenamos las noticias a 200
palabras

trunc_type = 'post'
padding_type = 'post'

tokenizer = Tokenizer(num_words=vocab_size, oov_token="<OOV>")
tokenizer.fit_on_texts(df['news'])

# Convertimos texto a secuencias de números
sequences = tokenizer.texts_to_sequences(df['news'])
# Aseguramos que todas midan lo mismo
padded = pad_sequences(sequences, maxlen=max_length,
padding=padding_type, truncating=trunc_type)
```

- **¿Qué hace?:** Si una noticia es "Goles en el estadio", el Tokenizer la vuelve [45, 12, 5, 89]. El Padding la convierte en [45, 12, 5, 89, 0, 0, 0...] hasta llegar a 200.

Bloque 3: División de Datos

```
X_train, X_test, y_train, y_test = train_test_split(padded,
df['label_num'], test_size=0.2, random_state=42)
```

Bloque 4: Creación de la Red Neuronal

Aquí definimos la arquitectura. Usaremos una capa **Embedding**, que es donde ocurre la "magia" de entender el significado de las palabras.

```
model = tf.keras.Sequential([
    # 1. Capa de Embedding: Crea un espacio vectorial para las
    palabras
    tf.keras.layers.Embedding(vocab_size, 16,
    input_length=max_length),

    # 2. Capa GlobalAveragePooling: Reduce la dimensionalidad
    tf.keras.layers.GlobalAveragePooling1D(),

    # 3. Capa Densa: Una capa intermedia de neuronas para aprender
    patrones
    tf.keras.layers.Dense(24, activation='relu'),

    # 4. Capa de Salida: Una neurona por cada categoría (deportes,
    política, etc.)
    tf.keras.layers.Dense(num_clases, activation='softmax')
])

model.compile(loss='sparse_categorical_crossentropy',
optimizer='adam', metrics=['accuracy'])
model.summary()
```

Bloque 5: Entrenamiento

En Deep Learning, el modelo no aprende de un solo golpe (como `fit` en Sklearn), sino que repite el proceso varias veces (**epochs**).

```
history = model.fit(
    X_train, y_train,
    epochs=10,
    validation_data=(X_test, y_test),
    verbose=2
)
```

Bloque 6: Evaluación y Predicción

```
# Evaluamos la precisión final
loss, accuracy = model.evaluate(X_test, y_test)
print(f"Precisión del modelo Deep Learning: {accuracy*100:.2f}%")

# Ejemplo de predicción con una noticia nueva
nueva_noticia = ["El equipo ganó el campeonato de liga en el último minuto"]
secuencia_nueva = tokenizer.texts_to_sequences(nueva_noticia)
padded_nueva = pad_sequences(secuencia_nueva, maxlen=max_length)

prediccion = model.predict(padded_nueva)
clase_predicha =
le.inverse_transform([tf.argmax(prediccion[0]).numpy()])
print(f"La noticia se clasifica como: {clase_predicha[0]}")
```

Resumen de lo que obtienes:

- **loss**: Es un número que indica qué tan "lejos" está la predicción de la realidad (cuanto más cerca de 0, mejor). Se usa para que el modelo aprenda.
- **accuracy**: Es el porcentaje de aciertos (ej: 0.85 significa 85% de noticias clasificadas correctamente). Es lo que nosotros usamos para entender si el modelo es bueno.

¿Qué cambió respecto a Scikit-learn?

1. **Representación de palabras**: En lugar de solo contar si la palabra "gol" aparece (ML clásico), el **Embedding** del Deep Learning coloca palabras con significados similares (como "gol", "estadio", "partido") "cerca" unas de otras en un mapa matemático.
2. **Arquitectura flexible**: En Deep Learning puedes añadir más capas (neuronas) para que el modelo aprenda relaciones más complejas.
3. **Procesamiento**: No necesitamos necesariamente Stemming o Lematización, ya que el modelo puede aprender que "corriendo" y "correr" tienen vectores similares si ve suficientes ejemplos.
4. **Escalabilidad**: Este enfoque funciona mucho mejor cuando tienes cientos de miles de noticias.